Project Management Conventional Software Management

Vivek Gupta
Asstt. Prof. Dept of CSE

Introduction to Project Management

Project :

Converting a vision, a dream or a need to reality.

- ✓ A job that has a beginning and an end (Time)
- ✓ A specified outcome(Scope)
- ✓ At a stated level of Performance (Quality)
- ✓ At a budget(Costs).

Project Characteristics :

- *Temporary* : Has definite Start and Finish
- *Unique* : Product/Service is different in some distinguishing way

Introduction Continues…

- Management: Management is the technique of understanding the problems, needs and controlling the use of Resources, Cost, Time, Scope and Quality.
- Project Management : Application of knowledge, skills , tools & techniques to project activities in order to meet stakeholder needs & expectations from a project.
- Needs : stated part of the project
- Expectations : unstated part of the project
- "Completion of Project on time within Budget without comprising Quality"

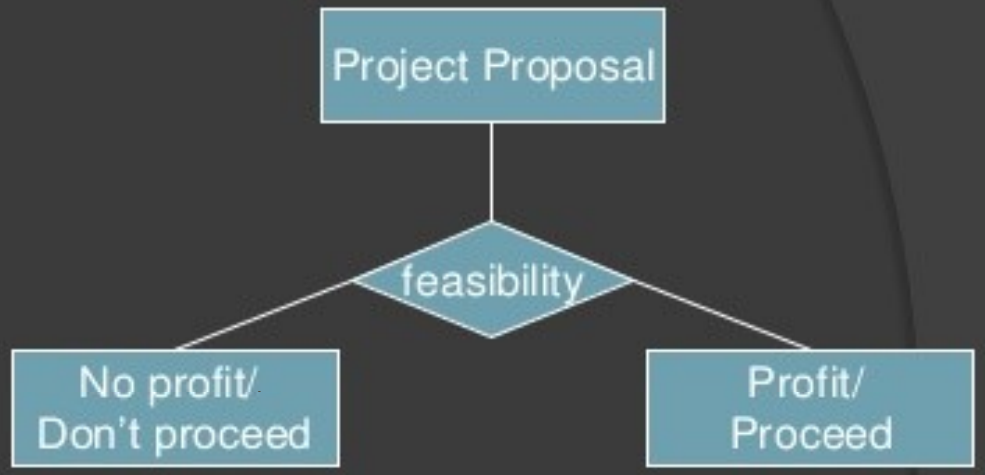
Why do Companies use PM

- To handle projects effectively in an organization.
- To define the project and agree with the customer
- To plan and assess resource needs for the project
- To estimate project cost and make proposals
- To plan & schedule activities in a project.
- To allocate the right resource at the right time.
- To assess risk and failure points and make backup plans.
- To lead a project team effectively and communicate well among team members.

Evolution of Software Economics

IMPORTANCE

- Feasibility analysis.



- ROI (Return Over Investment).

2.1 Software Economics

- Five fundamental parameters that can be abstracted from software costing models:
  - Size
  - Process
  - Personnel
  - Environment
  - Required Quality

Software Economics – Parameters (1 of 4)

- Size: Usually measured in SLOC or number of Function Points required to realize the desired capabilities.
  - Function Points – a better metric earlier in project
  - LOC (SLOC, KLOC…) a better metric later in project
  - software metric used to measure the amount of code in a software program
  - Breaks the system into smaller components so they can be better understood and analyzed.
- Process – used to guide all activities.
  - Workers (roles), artifacts, activities…
  - Support heading toward target and eliminate non-essential / less important activities
  - Process **critical** in determining software economics
    - Component-based development; application domain…iterative approach, use-case driven…
  - Movement toward '**lean**'[23]… everything!

Software Economics – Parameters (2 of 4)

- Personnel – capabilities of the personnel in general and in the application domain in particular
  - Motherhood: get the right people; good people; Can't always do this.
  - Much specialization nowadays. Some are terribly expensive.
  - Emphasize 'team' and team responsibilities…Ability to work in a team;
    - Several newer light-weight methodologies are totally built around a team or very small group of individuals…

Software Economics – Parameters (3 of 4)

- Environment – the tools / techniques / automated procedures used to support the development effort.
  - Integrated tools; automated tools for modeling, testing, configuration, managing change, defect tracking, etc…
- Required Quality – the functionality provided; performance, reliability, maintainability, scalability, portability, user interface utility; usability…

Software Economics – Parameters (4 of 4)

Effort = (personnel)(environment)(quality)(size ^{Process})

(Note: effort is exponentially related to size….)

What this means is that a 10,000 line application will cost less per line than a 100,000 line application.

- These figures – surprising to the uninformed – are true.
- Fred Brooks – Mythical Man Month – cites over and over that the additional communications incurred when adding individuals to a project is very significant.
  - Tend to have more reviews, meetings, training, biases, getting people up to speed, personal issues…
- Let's look at some of the trends:

Notice the Process Trends….for three generations of software economics

- Conventional development (60s and 70s)
  - Application – custom; Size – 100% custom
  - Process – ad hoc …(discuss) – laissez faire;
  - 70s - SDLC; customization of process to domain / mission, structured analysis, structured design, code.
- Transition (80s and 90s)
  - Environmental/tools – some off the shelf.
    - Tools: separate, that is, often not integrated esp. in 70s…
  - Size: 30% component-based; 70% custom
  - → Process: repeatable
- Modern Practices (2000 and later)
  - Environment/tools: off-the-shelf; integrated
  - Size: 70% component-based; 30% custom
  - Process: managed; measured (refer to CMM)

Notice Performance Trends….for three generations of software economics

- Conventional: Predictably bad: (60s/70s)
  - usually always over budget and schedule; missed requirements
    - All custom components; symbolic languages (assembler); some third generation languages (COBOL, Fortran, PL/1)
    - Performance, quality almost always less than great.
- Transition: Unpredictable (80s/90s)
  - Infrequently on budget or on schedule
  - Enter software engineering; 'repeatable process;' **project management**
  - Some commercial products available – databases, networking, GUIs; But with huge growth in complexity, (especially to distributed systems) existing languages and technologies not enough for desired business performance
- Modern Practices: Predictable (>2000s)
  - Usually on budget; on schedule. Managed, measured **process management**. Integrated environments; 70% off-the-shelf components, Component-based applications RAD; iterative development; stakeholder emphasis.

2.2 "Pragmatic" Software Cost Estimation

- Little available on estimating cost for projects using iterative development.
  - Difficult to hold all the controls constant
    - Application domain; project size; criticality; etc. Very 'artsy.'
    - Metrics (SLOC, function points, etc.) NOT consistently applied EVEN in the same application domain!
    - Definitions of SLOC and function points are not even consistent!
  - Much of this is due to the nature of development. There is no magic date when design is 'done;' or magic date when testing 'begins' …
  - Consider some of the issues:

Cost Estimation

- Estimation involves answering the following questions:
  - How much effort is required to complete each activity?
  - How much calendar time is needed to complete each activity?
  - What is the total cost of each activity?
- Project cost estimation and project scheduling are usually carried out together.

Development Cost

- The costs of development are primarily the costs of the effort involved, so the effort computation is used in both the cost and the schedule estimate.
- The initial cost estimates may be used to establish a budget for the project and to set a price for the software for a customer.
- The total cost of a software development project is the sum of following costs:
  - Hardware and software costs including maintenance.
  - Travel and training costs.
  - Effort costs of paying software developers.
-

Effort Cost

- For most projects, the dominant cost is the effort cost.
- Effort costs are not just the salaries of the software engineers who are involved in the project.
- The following overhead costs are all part of the total effort cost:
  - Costs of heating and lighting offices.
  - Costs of support staff (accountants, administrators, system managers, cleaners, technicians etc.).
  - Costs of networking and communications.
  - Costs of central facilities (library, recreational facilities, etc.).
  - Costs of social security and employee benefits such as pensions and health insurance.

Three Issues in Software Cost Estimation:

- 1. Which cost estimation model should be used?
- 2. Should software size be measured using SLOC or Function Points? (there are others too…)
- 3. What are the determinants of a good estimate? (How do we know our estimate is good??)

So very much is dependent upon estimates!!!!

Cost Estimation Models

- Many available.
- Many organization-specific models too based on their own histories, experiences…
  - Oftentimes, these are super if 'other' parameters held constant, such as process, tools, etc. etc.
- COCOMO, developed by Barry Boehm, is the most popular cost estimation model.
- Two primary approaches:
  - Source lines of code (SLOC) and
  - Function Points (FP)

Source Lines of Code (SLOC)

- Many feel comfortable with 'notion' of LOC
- SLOC has great value – especially where applications are custom-built.
  - Easy to measure & instrument – have tools.
  - Nice when we have a history of development with applications and their existing lines of code and associated costs.
- Today – with use of components, source-code generation tools, and objects have rendered SLOC somewhat ambiguous.
  - We often don't know the SLOC – but do we care? How do we factor this in? →

Source Lines of Code (SLOC)

- Generally more useful and precise basis than FPs
- Appendix D – an extensive case study.
  - Addresses how to count SLOC where we have reuse, different languages, etc.
- We will address LOC in much more detail later.
- Appendix provides hint at the complexity of using LOC for software sizing particularly with the new technologies using automatic code generation, components, development of new code, and more.

Function Points

- Use of Function Points - many proponents.
  - International Function Point User's Group – 1984 – "is the dominant software measurement association in the industry."
  - Check out their web site (www.IFPUG.com ??)
  - Tremendous amounts of information / references
  - Attempts to create industry standards….

- → Major advantage: Measuring with function points is independent of the technology (programming language, tools …) used and is thus better for comparisons among projects. →

Function Points

- Function Points measure numbers of
  - external user inputs,
  - external outputs,
  - internal data groups,
  - external data interfaces,
  - external inquiries, etc.
- ➔ Major disadvantage: Difficult to measure these things.
  - Definitions are primitive and inconsistent
  - Metrics difficult to assess especially since normally done earlier in the development effort using more abstractions.
- Yet, no project will be started without estimates!!!!

23

But:

- Cost estimation is a real necessity!!! Necessary to 'fund' project!
- All projects require estimation in the beginning (inception) and adjustments…
  - These must stabilize; They are rechecked…
  - Must be **reusable** for additional cycles
  - Can create organization's own methods of measurement on how to 'count' these metrics...
- No project is arbitrarily started without cost / schedule / budget / manpower / resource estimates (among other things)
- ➔ SO critical to budgets, resource allocation, and to a host of stakeholders

So, How Good are the Models?

- COCOMO is said to be 'within 20%' of actual costs '70% of the time.' (COCOMO has been revised over the years…)
- Cocomo (Constructive Cost Model) is a regression model based on LOC, i.e **number of Lines of Code**.
- It is a procedural cost estimate model for software projects.
- It was proposed by Barry Boehm in 1970 and is based on the study of 63 projects, which make it one of the best-documented models.
- Yet, all non-trivial software development efforts require costing; It is a basic management activity.

- So, let's look at top down and bottom up estimating.

Types of Cocomo Models

- COCOMO consists of a hierarchy of three increasingly detailed and accurate forms.
- Any of the three forms can be adopted according to our requirements.
- These are types of COCOMO model:
  - Basic COCOMO Model
    - can be used for quick and slightly rough calculations of Software Costs.
    - Its accuracy is somewhat restricted due to the absence of sufficient factor considerations.
  - Intermediate COCOMO Model
    - takes these Cost Drivers into account
  - Detailed COCOMO Model
    - additionally accounts for the influence of individual project phases, i.e in case of Detailed it accounts for both these cost drivers and also calculations are performed phase wise henceforth producing a more accurate result.

Top Down versus Bottom Up Substantiating the Cost…

- Most estimators perform bottom up costing - **substantiating** a target cost - rather than approaching it a top down, which would yield a 'should cost.'
- Many project managers create a 'target cost' and then play with parameters and sizing until the target cost can be justified…
  - Work backwards!
  - Attempts to win proposals, convince people, …
- Any approach should force the project manager to assess risk and discuss things with stakeholders…

Top Down versus Bottom Up

- Bottom up … substantiating? Good?
  - If well done, it requires considerable analysis and expertise based on much experience and knowledge;
    Development of similar systems a great help; similar technologies…
  - If not well done, causes team members to go ***crazy***! (This is not uncommon)
- Independent cost estimators (consultants…) not reliable.

Author suggests:

- Likely best cost estimate is undertaken by an **experienced project manager**, software architect, developers, and test managers – and this process can be quite iterative!
- **Previous experience is essential**. Risks identifiable, assessed, and factored in.
- When created, the **team must live with** the cost/schedule **estimate**.
- More later in course. But for now→ (Heuristics from our text:)

A Good Project Estimate:

- ➔Is conceived and supported by the project manager, architecture team, development team, and test team accountable for performing the work.
- ➔Is accepted by all stakeholders as ambitious but doable
- Is based on a well-defined software cost model with a credible basis
- ➔ Is based on a database of relevant project experience that includes similar processes, similar technologies, similar environments, similar quality requirements, and similar people, and
- ➔ Is defined in enough detail so that its key risk areas are understood and the probability of success is objectively assessed.

A Good Project Estimate

- Quoting: "An 'ideal estimate' would be derived from a mature cost model with an experience base that reflects multiple similar projects done by the same team with the same mature processes and tools.

- "Although this situation rarely exists when a project team embarks on a new project, good estimates can be achieved in a straightforward manner in later life-cycle phases of a mature project using a mature process."